Number and Group Theory

TC50331E

Algebraic Coding Theory

Jaish Singh
33139652

April 2026

1 Block Codes

Let $\mathbf{B}(n) = \mathbb{Z}_2^n$, the set of all binary strings of length n. That is,

$$\mathbf{B}(n) = \{(x_1, \ldots, x_n) \mid x_i \in \mathbb{Z}_2 \text{ for } 1 \leq i \leq n\}.$$

with a binary component-wise addition modulo 2: $(x_1, \ldots, x_n) \oplus (y_1, \ldots, y_n) = (x_1 + y_1, \ldots, x_n + y_n)$, where addition is taken in $\mathbb{Z}_2$. This operation is well-defined since $\mathbb{Z}_2$ is closed under addition. We verify the group axioms for $(\mathbf{B}(n), \oplus)$:

1. *Closure:* Let $x, y \in \mathbf{B}(n)$. Then $x \oplus y \in \mathbf{B}(n)$ since addition in $\mathbb{Z}_2$ is closed.
2. *Associativity:* For all $x, y, z \in \mathbf{B}(n)$, we have

   $$(x \oplus y) \oplus z = x \oplus (y \oplus z),$$

   since addition in $\mathbb{Z}_2$ is associative.
3. *Identity:* The element $0 = (0, \ldots, 0) \in \mathbf{B}(n)$ satisfies $x \oplus 0 = x$ for all $x \in \mathbf{B}(n)$.
4. *Inverses:* For each $x \in \mathbf{B}(n)$, we have $x \oplus x = 0$, so every element is its own inverse.

Having established that $(\mathbf{B}(n), \oplus)$ is a group, we now observe that it is in fact abelian. This follows from the commutativity of addition in $\mathbb{Z}_2$, applied component-wise. Since digital signals are inherently binary, it is natural to model transmitted data as elements of $\mathbb{Z}_2 = \{0, 1\}$, the group of integers modulo 2 under addition. The group operation, which coincides with XOR, captures parity computations precisely: the parity of a binary string is simply its sum in $\mathbb{Z}_2$. This structure extends naturally to $\mathbb{Z}_2^n$, the group of binary strings of length n, which serves as the space in which code words live.

For $0 < k < n$, we define an (n, k) *block code* to be a subgroup of $\mathbf{B}$(n), such that $|(n, k)| = 2^k$, where elements are binary strings, called *code words*, of length n. Code words are the only strings allowed to be transmitted. Due to channel errors where the bits flip to the opposite state during transmission, it is not always guranteed that the same code word is received by the client, so the *received words* are elements of a broader set of all binary strings of length n, $\mathbf{B}$(n).

Consider $C = \{(0, 0, 0, 0), (0, 1, 0, 1), (1, 0, 1, 0), (1, 1, 1, 1)\}$. For all $a, b \in C$, $a - b \in C$, which establishes that C is a subgroup of $\mathbf{B}(4)$. More specifically, $|C| = 2^2$ so therefore C is a (4, 2) block code. The received word for C can be any binary string of length 4, i.e. any element in $\mathbf{B}(4) =$ {0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111}.

To systematically generate code words in a block code, we use a *generator matrix* [1]. A generator matrix for an (n, k) block code C is a $k \times n$ matrix G whose rows form a basis for C, i.e. are linearly independent. For instance, the generator matrix G, for a (5, 2) block code $C = \{(0,0,0,0,0), (1,0,1,1,0), (0,1,1,0,1), (1,1,0,1,1)\}$ is

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

To verify if a received word is a valid code word, we make use of a $(n-k) \times n$ *parity-check matrix*, **H**. The valid code words are the null space of the parity-check matrix, i.e.,

$$HG^T = 0$$

Similarly, a received word C is a valid code word if it satisfies the equation $HC^T = 0$. The parity-check matrix, H, for our (5, 2) block code is

$$H = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

$$HG^T = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

For a given block code, we must define it's capabilities in terms of error detection and error correction. This capability is defined in terms of the *weight* of a code word in a block code. The weight of a binary string $b \in \mathbf{B}(n)$, denoted by $w(b)$, is the count of 1s in b. For instance, the weight of $(1,0,1,1,0,1,1,0,1,1,0,1) \in \mathbf{B}(12)$ is 8. The *distance* D, between two binary strings, $x, y \in \mathbf{B}(n)$, is the weight of their sum in $\mathbb{Z}_2$, i.e.,

$$D(x, y) = w(x \oplus y)$$

In other words, the distance is the number of positions where the two strings differ. The distance defines a metric [3] on **B**(n). Formally, a metric defined on a set is a function such that it follows

- Non-degeneracy: $d(x, y) = 0 \iff x = y$
- Symmetry: $d(x, y) = d(y, x)$
- Triangle inequality: $d(x, z) \leq d(x, y) + d(y, z)$

It is straightforward to verify that D satisfies each of these.

- Non-degeneracy: $D(x, y) = w(x \oplus y) = 0$ if and only if $x \oplus y = 0$, which holds if and only if $x_i = y_i$ for all i, i.e. $x = y$.
- Symmetry: $D(x, y) = w(x \oplus y) = w(y \oplus x) = D(y, x)$, since $x_i \oplus y_i = y_i \oplus x_i$ for each bit.
- Triangle inequality: For any $x, y, z \in \mathbf{B}(n)$, note that $x \oplus z = (x \oplus y) \oplus (y \oplus z)$, so

$$\begin{aligned} D(x, z) &= w(x \oplus z) \\ &= w\big((x \oplus y) \oplus (y \oplus z)\big) \leq w(x \oplus y) + w(y \oplus z) \\ &= D(x, y) + D(y, z) \end{aligned}$$

  where the inequality uses subadditivity of weight. To see subadditivity holds, observe that $(a \oplus b)_i = 1$ only if $a_i = 1$ or $b_i = 1$, so every bit counted in $w(a \oplus b)$ is counted in $w(a) + w(b)$.

This metric structure underlies the error-correcting power of block codes, which we explore further in the following section.

2 Decoding and Errors

Let C be a code word to be transmitted, whilst R is its received counterpart. The error in transmission is measured by the distance between the sent binary string and the received string, $D(C, R)$. These errors change the message seemingly at random coordinates, and therefore, there is a non zero chance for the received message to have been transformed to another valid code word. Let $d_{\min}$ be the smallest distance between any two valid code words in a block code, then the decoder can detect upto $d_{\min} - 1$ errors without coinciding with another code word. If R is detected to have errors, it may be possible to error correct the message. We simply do this by taking the nearest valid code word to R as the intended message.

If there is more than one nearest valid code word, the decoder cannot correct the message. Therefore, for there to be a unique valid code word for R, R must not be equidistant to two code words. Conversely, R must be at most $\lfloor \frac{d_{\min}-1}{2} \rfloor$ from a valid code word. Therefore, a code with minimum distance $d_{\min}$ can correct at most $\lfloor \frac{d_{\min}-1}{2} \rfloor$ errors, since any R within that radius is guaranteed to be closer to exactly one valid code word than any other. In other words, $d_{\min} \geq 2t+1$ [2], where t is the number of errors the block code can correct.

Let G be a block code described by

$$G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

where the code words for G is defined by the encoding algorithm

$$\mathbf{C} = \{B\mathbf{G} : B \in \mathbf{B}(3)\} \subset \mathbf{B}(6)$$
$$= \{000000,\ 100111,\ 010101,\ 001110,\ 110010,\ 101001,\ 011011,\ 111100\}$$

Since $\mathbf{C}$ is closed under addition in $\mathbb{Z}_2$ and contains 000000, we call C to be a linear code. As a result, for any two code words $C_1, C_2 \in \mathbf{C}$, their sum $C_1 \oplus C_2 \in \mathbf{C}$, and the distance satisfies $D(C_1, C_2) = w(C_1 \oplus C_2)$. Therefore, the smallest distance between any two valid code words, $d_{\min}$ would be the smallest weight of a codeword in C excluding 000000, which by inspection we can deduce to be 3, the weight of 010101. Using $d_{\min}$, we can infer that $\mathbf{G}$ can detect at most $d_{\min} - 1 = 2$ errors and correct $\lfloor \frac{d_{\min}-1}{2} \rfloor = 1$ error.

So far we have defined that the parity-check matrix, $\mathbf{H}$, detects if an input, $B \in \mathbf{B}(n)$, is valid or not by $F : \mathbf{B}(n) \to \mathbf{B}(n-k)$ given by

$$F(B) = \mathbf{H}B^t, B \in \mathbf{B}(n)$$

And if an error is detected, we take the nearest valid code word to be the message. However, to search for the nearest valid code word by individually measuring the distance between every possible word would be computationally expensive. To solve this issue, we use the fact that F is a surjective group homomorphism. We know that F is a group homomorphism, since it satisfies the condition $F(a + b) = F(a) + F(b)$, which follows from the linearity of matrix multiplication. We also know that $\mathbf{H}$ has a full row rank $n-k$ which follows from linear independence of $\mathbf{G}$. This suggests that the linear map F can produce any vector in $\mathbf{B}(n-k)$ and therefore is a surjective map. Thus, we can deduce that F is a surjective group homomorphism, which allows us to partition $\mathbf{B}(n)$ into cosets of the kernel of F.

$$F(B) = HB^t = H(C+e)^t = HC^t + He^t = 0 + He^t = He^t$$

where e is the error in B. We precompute all cosets of the kernel of F in $\mathbf{B}(n)$ and find the element with the smallest weight in each coset, called the coset leader, $\hat{e}$. When decoding, we find the coset

in which $F(B) = He^t$ belongs to in our precomputed set of cosets. Once we have our coset, and subsequently it's corresponding coset leader $\hat{e}$, finding the closest code word $\hat{C}$ becomes trivial

$$\hat{C} = B - \hat{e}$$

Since F is a group homomorphism, its kernel $ker(F) = \{B \in \mathbf{B}(n) : \mathbf{H}B^t = 0\}$ is a subgroup of $\mathbf{B}(n)$. Because $\mathbf{HG}^t$ was defined to be 0, all linear combinations of $\mathbf{G}$, i.e. all valid code words, is the kernel of F.

A natural question to ask at this point is what is the optimal value for k such that we maximise error-control capability whilst minimizing the number of redundant parity bits. This balance is what a *Hamming code* achieves. A Hamming code is a linear block code constructed so that the minimum distance between any two valid code words is 3, guaranteeing the ability to detect up to 2 errors and correct 1 error. This means the coset leaders are precisely the n single-bit error vectors together with the zero vector, giving $n + 1 = 2^m$ cosets partitioning $\mathbf{B}(n)$.

For a Hamming code, the parity-check matrix H is an $m \times n$ matrix whose columns consist of all $2^m - 1$ distinct non-zero binary vectors of length m. A received word B is a valid codeword if and only if $HB^T = 0$. If a single-bit error occurs at position i, the syndrome HB^t equals precisely the i-th column of H, which uniquely identifies the coset containing B. Since each single-bit error vector is the coset leader of a distinct coset, guaranteed by the fact that all columns of H are distinct and non-zero, we can recover $\hat{e}$ directly from the syndrome, making the lookup into our precomputed coset table trivial.

3 Emperical Investigation

We systematically vary n and k for a block code to observe changes in $d_{\min}$. We use $d_{\min}$ as a proxy of our block codes utility since both error correction as well as error detection capacity of a block code is derived from $d_{\min}$, where larger $d_{\min}$ relates to better error-control capabilities.

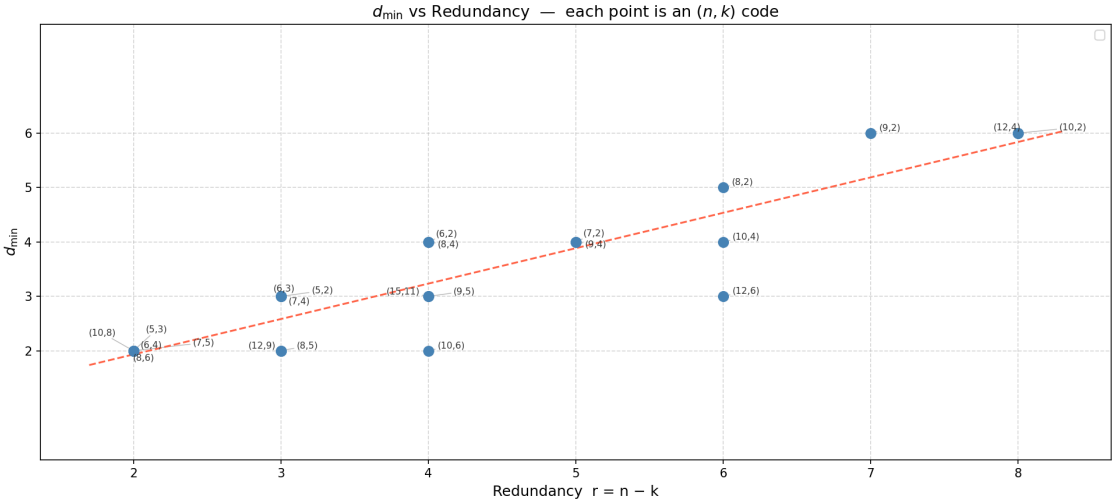

Figure 1: $d_{\min}$ vs Redundancy for varying block codes

The scatter plot confirms a clear positive trend between redundancy $r = n - k$ and $d_{\min}$: as more redundant bits are added, higher minimum distances become achievable, consistent with the Singleton bound. However, the significant vertical spread at each value of r suggests that redundancy is a necessary but not sufficient condition for large $d_{\min}$, the specific code construction is equally important. For example, at r = 4, $d_{\min}$ ranges from 2 for the (10,6) code to 4 for the (6,2) code. The (15,11) Hamming code is a notable point of interest: it achieves $d_{\min} = 3$ at r =

4 while encoding 11 information bits, highlighting that well-structured codes can maintain useful error-correction capability without sacrificing information rate. The dashed trend line captures the general relationship, but the scatter around it emphasizes that choosing a block code requires optimising construction, not just increasing redundancy.

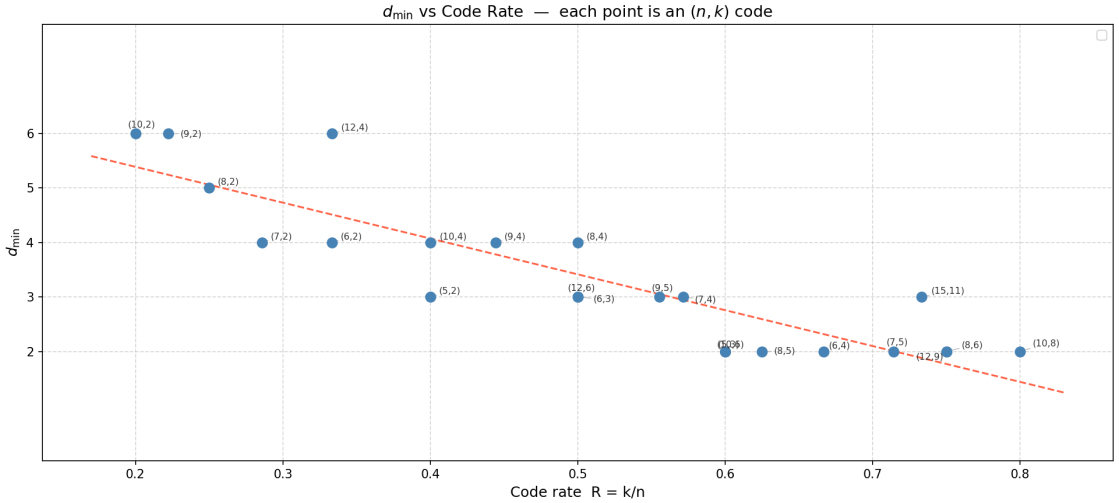

Figure 2: $d_{\min}$ vs Code Rate for varying block codes

Figure 2 confirms the fundamental antagonism between code rate R = k/n and error-correction strength: as R increases, the achievable $d_{\min}$ falls, consistent with the Plotkin and Singleton bounds. The negative trend is clear, but the scatter around it again shows that rate alone does not fix $d_{\min}$, code construction remains decisive. The (12,4) code is a notable outlier, achieving $d_{\min} = 6$ at $R \approx 0.33$, well above comparable code rates, illustrating how longer block lengths can extract greater distance at the same rate. At the high-rate end, the (15,11) Hamming code stands out for sustaining $d_{\min} = 3$ at $R \approx 0.73$, a region where most codes are limited to $d_{\min} = 2$. A pronounced floor at $d_{\min} = 2$ emerges for $R \geq 0.6$, suggesting that beyond this efficiency threshold, further gains in rate come essentially for free in terms of error-correction degradation, the capability is already at its practical minimum. The widest vertical spread occurs at low rates, indicating that low-rate codes offer the greatest design latitude, while high-rate codes converge tightly toward the minimum distance floor.

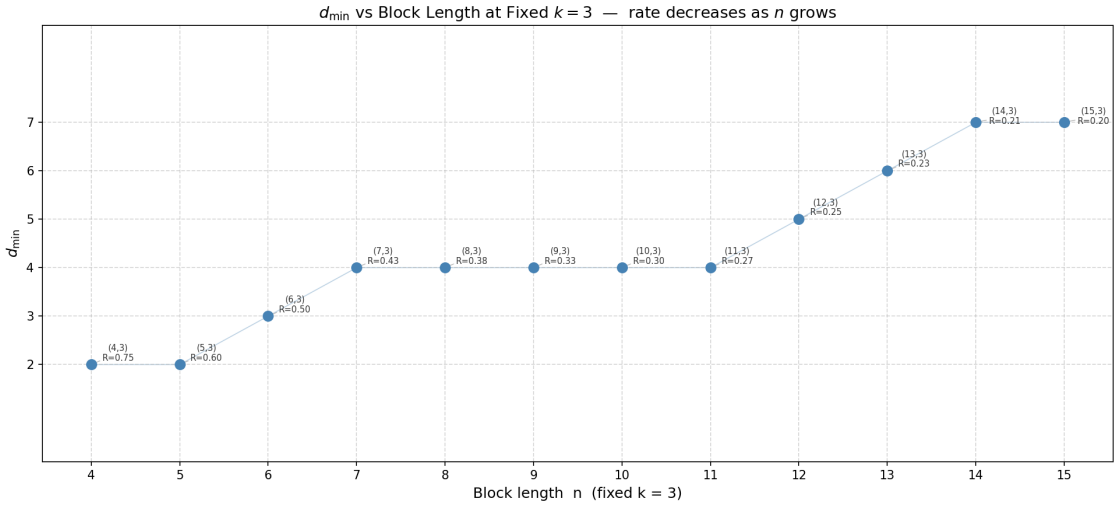

Figure 3: $d_{\min}$ vs n for fixed k

With k fixed at 3, increasing the block length n is equivalent to adding redundancy while holding information bits constant, and Figure 3 shows that this reliably grows $d_{\min}$, but not linearly. The

progression is distinctly step-wise, $d_{\min}$ increases' are separated by flat plateaus, reflecting the discrete geometry of Hamming space rather than a continuous trade-off. The most notable plateau spans $n = 7$ to $n = 11$, where five codes share $d_{\min} = 4$ even as the code rate falls from $R = 0.43$ to $R = 0.27$, four extra redundant bits that deliver no improvement in error-correction capability. The breakout at $n = 12$ highlights that distance gains are unlocked only at specific thresholds, making naive redundancy addition an inefficient design strategy.

Figures 1-3 gives a coherent and practically important picture of block code design. Figure 1 establishes that redundancy is a prerequisite for large $d_{\min}$, but construction quality determines how efficiently that redundancy is converted. Figure 2 reframes the same trade-off through code rate, revealing a $d_{\min}$ floor at high rates and the greatest design freedom at low rates. Figure 3 isolates block length, even with k fixed, increases in $d_{\min}$ are not proportional to added redundancy, they arrive in discrete steps, with long plateaus where extra bits are effectively wasted. Across all three perspectives, the conclusion is the same, parameter choices set the ceiling on achievable $d_{\min}$, but it is the algebraic structure of the code that determines whether that ceiling is approached. Efficient block code design is therefore not just a matter of allocating more bits to redundancy, but of constructing codes whose geometry exploits every redundant bit to maximally separate codewords.

References

[1] ER Berlekamp. "A survey of coding theory". In: *Journal of the Royal Statistical Society Series A: Statistics in Society* 135.1 (1972), pp. 44–73.

[2] Raj Chandra Bose and Dwijendra K Ray-Chaudhuri. "On a class of error correcting binary group codes". In: *Information and control* 3.1 (1960), pp. 68–79.

[3] Richard W Hamming. "Error detecting and error correcting codes". In: *The Bell system technical journal* 29.2 (1950), pp. 147–160.

[4] Jaish Singh. *Algebraic Coding Theory, Coursework*. 2026. URL: https://github.com/jbsx/group_theory_coursework (accessed on 18/5/2026).

A Appendix

n	k	Redundancy $(n-k)$	code rate (k/n)	$d_{\min}$
5	2	3	0.40	3
5	3	2	0.60	2
6	2	4	0.33	4
6	3	3	0.50	3
6	4	2	0.67	2
7	2	5	0.29	4
7	4	3	0.57	3
7	5	2	0.71	2
8	2	6	0.25	5
8	4	4	0.50	4
8	5	3	0.63	2
8	6	2	0.75	2
9	2	7	0.22	6
9	4	5	0.44	4
9	5	4	0.56	3
10	2	8	0.20	6
10	4	6	0.40	4
10	6	4	0.60	2
10	8	2	0.80	2
12	4	8	0.33	6
12	6	6	0.50	3
12	9	3	0.75	2
15	11	4	0.73	3

Table 1: $d_{\min}$ for block codes of varying n and k [4]

n	k	Redundancy $(n-k)$	code rate (k/n)	$d_{\min}$
4	3	1	0.75	2
5	3	2	0.60	2
6	3	3	0.50	3
7	3	4	0.43	4
8	3	5	0.38	4
9	3	6	0.33	4
10	3	7	0.30	4
11	3	8	0.27	4
12	3	9	0.25	5
13	3	10	0.23	6
14	3	11	0.21	7
15	3	12	0.20	7

Table 2: $d_{\min}$ for fixed $k = 3$ and varying n [4]